Verification and Validation Report: AutoVox

Team #10, Five of a Kind
Omar Abdelhamid
Daniel Maurer
Andrew Bovbel
Olivia Reich
Khalid Farag

March 9, 2026

1 Revision History

Date	Version	Notes
March 9, 2026	1.0	Initial release

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test
TC	Test Case
FR	Functional Requirement
NF	Non-Functional Requirement
VnV	Verification and Validation
IM	Import Manager
VM	Visualization Manager
EM	Editing Manager
XM	Export Manager
MAX_VOXELS	Maximum number of voxels to display in the 3D view
STL	Standard Tessellation Language

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**
3.1 CAD File Import and Project Management Tests (Import Manager) 1
3.2 Model Configuration and Slicing Tests (Import Manager) 2
3.3 3D Model Rendering and Display Tests (Visualization Manager) 2
3.4 Layer Focus and Voxel Selection Tests (Visualization Manager) 3
3.5 Material and Magnetization Tracking Tests (Visualization Manager) 3
3.6 Integration Tests (Import to Visualization) 4
3.7 Editing Manager Tests 4
3.8 Export Manager Tests 5

4 Nonfunctional Requirements Evaluation **6**
4.1 Usability 6
4.2 Performance 6
4.3 etc. 6

5 Comparison to Existing Implementation **6**

6 Unit Testing **7**
6.1 Import Manager Tests 7
6.1.1 CAD File Import and Project Management 7
6.1.2 Model Configuration and Slicing 8
6.2 Visualization Manager Tests 9
6.2.1 3D Model Rendering and Display 9
6.2.2 Layer Focus and Voxel Selection Tests 10
6.2.3 Material and Magnetization Tracking 12
6.3 Editing Manager Tests 12
6.3.1 Editing a voxel 13
6.3.2 Utilizing the history manager 17
6.3.3 Saving project 19
6.4 Export Manager Tests 20
6.4.1 Export Validation and File Export 20

7 Changes Due to Testing **22**
7.1 Usability and User Input Adjustments 22
7.2 Interaction Flow Adjustments 23
7.3 Performance Adjustments 23
7.4 Future Improvements 23

8 Automated Testing **24**

9 Trace to Requirements **24**
9.1 Functional Requirements . . . 24
9.2 Non-Functional Requirements . . . 28

10 Trace to Modules **29**

11 Code Coverage Metrics **33**
11.1 Python Code Coverage . . . 33
11.2 TypeScript Code Coverage . . . 33

List of Tables

1 Import Manager: STL File Import and Project Management Test Cases . . . 7
2 Import Manager: Model Configuration and Slicing Test Cases . . . 8
3 Visualization Manager: 3D Model Rendering and Display Test Cases . . . 10
4 Visualization Manager: Layer Focus and Voxel Selection Test Cases . . . 11
5 Visualization Manager: Material and Magnetization Tracking Test Cases . . . 12
6 Editing Manager: Editing a voxel (system and backend) . . . 13
7 Editing Manager: Utilizing the history manager (backend) . . . 18
8 Editing Manager: Saving project (backend) . . . 20
9 Export Manager Test Cases . . . 21
10 Trace Between Import Manager Test Cases and Corresponding Requirements 25
11 Trace Between Visualization Manager test cases and corresponding requirements 25
12 Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 1: voxel editing and history) . . . 26
13 Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 2: layers, add/delete, reset, save) . . . 27
14 Trace Between Export Manager Test Cases and Corresponding Requirements 27
15 Trace Between Non-Functional Requirements Import Manager Test Cases and Corresponding Requirements . . . 28
16 Trace Between Non-Functional Requirements Visualization Manager Test Cases and Corresponding Requirements . . . 28
17 Trace Between Non-Functional Requirements Editing Manager Test Cases and Corresponding Requirements . . . 28
18 Trace Between Non-Functional Requirements Export Manager Test Cases and Corresponding Requirements . . . 29
19 Trace Between Import Manager Test Cases and Modules . . . 29
20 Trace Between Visualization Manager Test Cases and Modules . . . 30
21 Trace Between Editing Manager Test Cases and Modules (Part 1: voxel editing and history) . . . 31
22 Trace Between Editing Manager Test Cases and Modules (Part 2: layers, add/delete, reset, save) . . . 32
23 Trace Between Export Manager Test Cases and Modules . . . 32

List of Figures

1 Python Code Coverage 33
2 TypeScript Code Coverage 34

This document presents the Verification and Validation (VnV) Report for AutoVox. It summarizes the results of testing activities planned in the VnV Plan and demonstrates that the system satisfies the functional and non-functional requirements defined in the SRS. Detailed test-case tables and traceability are given in Section 6 and Section 9.
For detailed information about the project functional and non-functional requirements, please refer to the Software Requirements Specification document. For detailed information about the project modules, please refer to the Module Guide document. For detailed information about the project VnV Plan, please refer to the VnV Plan document.

3 Functional Requirements Evaluation

The following subsections report on each functional test case defined in the VnV Plan (Section 2 System Tests). For each test, the expected result is stated and the actual result is reported (pass or fail). Evidence is provided by backend unit tests (PyTest), frontend unit tests (Jest), and manual execution where applicable; see Section 6 and Section 8.

3.1 CAD File Import and Project Management Tests (Import Manager)

1. **test-FR-IM-1 Valid CAD File Import for New Project**

   The valid CAD file acceptance test ensures that the system correctly processes a syntactically valid STL file and creates a new project with the imported model ready for voxel slicing and visualization. A valid STL file was provided as input through the import interface, and the expected result was that the system should accept the file, create the project, and initiate the voxel slicing process. The actual result confirmed that the system successfully accepted the file and created the project; evidence is provided by backend and frontend unit tests (e.g., application load and model list, fetch available models).

2. **test-FR-IM-2 Past Project File Import**

   This test verifies that the system correctly loads a previously saved project file with magnetization and material properties preserved. A saved project file was selected from the file/project list, and the expected result was that the system loads the project and preserves all properties. The actual result confirmed that the system correctly loaded the project and voxel data; partitions and voxelized coordinates were fetched as expected (frontend and backend unit tests passed).

3. **test-FR-IM-3 Invalid File Format Handling**

   The invalid file format test ensures that the system rejects unsupported file types and provides clear feedback. A non-STL file (e.g., .txt, .jpg, .pdf) was provided as input, and the expected result was that the system should reject the file and display an error indicating unsupported format. The actual result confirmed that the system correctly rejected non-.stl files and displayed the appropriate alert (frontend unit test passed).

3.2 Model Configuration and Slicing Tests (Import Manager)

1. **test-FR-IM-4 Voxel Size Configuration**

   This test ensures that the system applies user-specified voxel dimensions and slices the model into voxels of the requested size. Custom voxel dimensions were provided as input, and the expected result was that the system applies the dimensions and slices the model accordingly. The actual result confirmed that the system correctly applied voxel dimensions and slicing (backend unit test passed).

2. **test-FR-IM-5 Model Scaling Functionality**

   This test verifies that the system scales the model to the desired printing size before slicing while maintaining geometric integrity. Model dimensions were modified via the interface, and the expected result was that the model is scaled appropriately. The actual result confirmed that the system correctly applied model scaling (backend unit test passed).

3. **test-FR-IM-6 Partial Voxel Resolution**

   This test ensures that the system correctly resolves partial voxels and preserves model integrity when surfaces intersect voxel boundaries. A model with such geometry was imported, and the expected result was correct resolution and preservation of accuracy. The actual result confirmed correct behaviour (backend unit test passed).

4. **test-FR-IM-7 Model Division for Display**

   This test verifies that when a model contains more than MAX_DISPLAY voxels, the system partitions it into manageable display sections. A large model was imported, and the expected result was partitioning without exceeding the threshold. The actual result confirmed that the system correctly partitioned the model (backend unit tests passed).

3.3 3D Model Rendering and Display Tests (Visualization Manager)

1. **test-FR-VM-1 3D Model Rendition with Clear Voxel Visualization**

   This test ensures that the Visualization Manager renders a 3D model with clear visualization of individual voxels and distinct boundaries. Sliced voxel data was provided to the viewer, and the expected result was correct rendering with status reporting (e.g., loading then ready). The actual result confirmed that the ModelViewer component renders and invokes voxel rendering utilities correctly (frontend unit tests passed).

2. **test-FR-VM-2 Partition Navigation Functionality**

   This test verifies that users can navigate between model partitions and access all display sections. Partition selection was exercised via the Partitions panel (open, select partition, verify selected state). The expected result was that the system fetches partitions and allows selection. The actual result confirmed that `fetchPartitions` was

called, partition names were displayed, and `onPartitionSelect` was invoked correctly (frontend unit tests passed).

3. **test-FR-VM-3 Multi-Perspective 3D Model Interaction**

   This test ensures that the system provides intuitive rotate and zoom controls for multi-perspective viewing. Interface controls for rotation and zoom were used; the expected result was responsive interaction. The actual result confirmed that the viewer component and controls behave as required (supported by frontend and manual testing).

3.4 Layer Focus and Voxel Selection Tests (Visualization Manager)

1. **test-FR-VM-4 Layer Isolation Functionality**

   This test verifies that users can isolate a specific layer to facilitate property assignments. The Layer Editor was opened with a project and partition, and layer data was fetched; the expected result was that the selected layer is isolated and the layer editor displays layers. The actual result confirmed that `fetchLayers` was called with correct parameters and the Layer Editor and Layer2DGrid displayed canvas and layer navigation (frontend unit tests passed).

2. **test-FR-VM-5 Voxel Selection Highlighting**

   This test ensures that the system provides clear visual feedback for which voxels are selected within a layer. Click and Lasso mode buttons were used and canvas selection was exercised; the expected result was distinct highlighting and correct mode behaviour. The actual result confirmed that Click and Lasso modes and selection handling work as specified (frontend unit tests passed).

3.5 Material and Magnetization Tracking Tests (Visualization Manager)

1. **test-FR-VM-6 Material Assignment Visual Tracking**

   This test verifies that voxel colours update to indicate material assignment completeness. Material IDs were assigned to voxels in the layer editor; the expected result was updated colours and clear feedback. The actual result confirmed that material assignment and colour updates behave correctly (frontend and backend unit tests passed).

2. **test-FR-VM-7 Magnetization Assignment Visual Tracking**

   This test ensures that voxels show magnetization assignment (e.g., arrow or indicator). Magnetization was assigned to voxels; the expected result was clear visual indication. The actual result confirmed correct behaviour (frontend and backend unit tests passed).

3.6 Integration Tests (Import to Visualization)

1. **test-FR-INT-1 Import to Visualization Data Transfer**

   This test ensures that when a valid CAD file is imported, the Visualization Manager receives complete voxel data and renders the 3D model without data loss or corruption. A valid CAD file was imported and the project was loaded; the expected result was successful transfer and rendering. The actual result confirmed that the application loads models, loads projects and voxel data, and the model viewer receives and renders data correctly (frontend and backend unit tests passed).

3.7 Editing Manager Tests

1. **test-FR-EM-1 Individual Voxel Magnetization Assignment** and **test-FR-EM-2 Group Voxel Magnetization Assignment**

   These tests verify that the system correctly processes single and bulk magnetization assignment (F231). The expected result was that assigned magnetization is saved and visually updated. The actual result confirmed that the backend saves magnetization and the UI supports assignment (backend unit tests passed).

2. **test-FR-EM-3 Favourite Magnetization Vector Management**

   This test ensures that users can create and reuse favourite magnetization vectors (F232). The expected result was that vectors are saved to the favourite bar and can be selected. The actual result confirmed correct behaviour (backend unit test passed).

3. **test-FR-EM-4** and **test-FR-EM-5 Individual and Group Voxel Material Assignment**

   These tests verify single and bulk material assignment (F233). The expected result was that material IDs are saved and displayed. The actual result confirmed correct behaviour (backend unit tests passed).

4. **test-FR-EM-6 Material Label Assignment**

   This test ensures that material IDs can be associated with labels (e.g., "Steel" for ID 1) (F234). The actual result confirmed correct behaviour (backend unit test passed).

5. **test-FR-EM-7 Property Replication Across Layers**

   This test verifies that properties can be copied from one layer to another (F235). The actual result confirmed correct behaviour (backend and frontend unit tests passed).

6. **test-FR-EM-8 Auto Save Functionality**

   This test ensures that the system automatically saves changes after property assignment (F236). The actual result confirmed that the project is saved to the database as expected (backend unit test passed).

7. **test-FR-EM-9 Edit History Access and Revert**

   This test verifies that users can undo and redo edits (F237). With a project loaded, Undo and Redo were triggered; the expected result was that the history API is called with the correct action. The actual result confirmed that `updateHistory` was called with `undo` and `redo` (frontend and backend unit tests passed).

8. **test-FR-EM-10 Entire Layer Selection and Assignment**

   This test verifies layer-level selection and assignment (F238). The actual result confirmed that layer list and layer access work correctly (backend unit tests passed).

9. **test-FR-EM-11** and **test-FR-EM-12 Manual Voxel Addition and Deletion**

   These tests ensure that users can add and remove voxels (F239). The actual result confirmed correct add/delete behaviour and error handling for non-existent voxels (backend and frontend unit tests passed).

10. **test-FR-EM-13** and **test-FR-EM-14 Individual and Group Voxel Property Reset**

    These tests verify that users can reset material and magnetization on selected voxels (F2310). The actual result confirmed correct reset behaviour (backend and frontend unit tests passed).

11. **test-FR-EM-15 Manual Save Functionality**

    This test ensures that the user can manually save the project (F2311). Save was triggered from the File menu (and via Ctrl+S); the expected result was that the download/save API is invoked and confirmation is shown. The actual result confirmed correct behaviour including no-project alert (frontend and backend unit tests passed).

3.8 Export Manager Tests

1. **test-FR-XM-1 Property Validation for Complete Magnetization** and **test-FR-XM-2 Property Validation for Incomplete Magnetization**

   These tests verify that the system validates magnetization (and material) completeness before export and prevents export of incomplete models (F241). The expected result was validation and appropriate errors or prompts. The actual result confirmed that validation and error handling work correctly (backend unit tests passed).

2. **test-FR-XM-3 Standalone File Export**

   This test verifies that the system can export a standalone file with all voxel metadata (F242). Export/CSV download was exercised; the expected result was that the file is created and saved. The actual result confirmed that the download API is called and blob handling works (frontend and backend unit tests passed).

3. **test-FR-XM-4 Project Export in Native Format**

This test ensures that users can export the project in native format (F243). The actual result confirmed correct export and error handling for non-existent or invalid project (backend unit tests passed).

4. **test-FR-XM-5 Export Progress Tracking**

   This test verifies that users receive feedback during export (F244). The actual result is supported by export flow and UI behaviour.

5. **test-FR-XM-6 Model Summary Export**

   This test ensures that the system can export a model with summary option (F245). The actual result confirmed correct behaviour (backend unit test passed).

6. **test-FR-XM-7 Export Failure Recovery**

   This test verifies that if export is interrupted, original project data remains intact (F241). The actual result confirmed that project data is not corrupted on simulated failure (backend unit test passed).

7. **test-FR-XM-8 Export Performance Validation**

   This test ensures that export meets the minimum rate threshold (F241). The actual result is supported by performance testing and backend validation.

Summary. All functional test cases from the VnV Plan listed above were executed; the actual results confirmed that the system behaviour matched the expected results (all passed). Detailed test-case tables with Ref. Req., Action, Expected Result, Actual Result, and Pass/Fail are in Section 6. Traceability from test cases to requirements is in Section 9.

4 Nonfunctional Requirements Evaluation

4.1 Usability

4.2 Performance

4.3 etc.

5 Comparison to Existing Implementation

AutoVox serves as an alternative to commercial multiphysics tools such as COMSOL Multiphysics® for workflows that involve editing and configuring voxel-based or mesh-based models. In the lab setting, AutoVox addresses several pain points that research assistants experience when using COMSOL for similar tasks.

COMSOL is a full-featured simulation platform, but manual editing across many voxels is tedious and error-prone. Research assistants report that the manual work is tiring and that saving behaviour is not ideal. When a mistake is made, they often must redo an entire section of work, because redo and undo are not very intuitive in the application. In addition,

crashes occur frequently, and COMSOL does not offer saved backups on a server, so recovery after a crash can mean lost work.

AutoVox is designed to improve this workflow by providing clearer redo and undo behaviour and by supporting bulk actions across many voxels. These features reduce the need to repeat large sections of manual edits when something goes wrong and make it easier to correct mistakes without starting over. The combination of intuitive undo/redo and server-backed or local project saving (where implemented) aims to make AutoVox a more forgiving and efficient tool for repetitive voxel-level editing in the research lab.

6 Unit Testing

This section reports **test results** and evidence for the system unit tests. The emphasis is on outcomes and evidence, not on the rationale for test selection (see the VnV Plan for that). Test cases are organized by manager: Import Manager (TC1–TC5), Visualization Manager (TC4–TC11), Editing Manager (TC12–TC61), and Export Manager (TC62–TC69).

For each test case, the tables below provide: **test case name** (TC id), **initial state** (implied by the Ref. Req. and test context), **input** (Action), **expected result**, and **whether actual output matched expected** (Result: Pass/Fail). Actual outcomes are backed by automated test runs; see Section 8 for how to reproduce.

6.1 Import Manager Tests

6.1.1 CAD File Import and Project Management

Table 1 gives the results for STL file import and project management tests (TC1–TC5).

Table 1: Import Manager: STL File Import and Project Management Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC1	F211	Provide a valid STL file to system through file settings panel	STL file is uploaded successfully to the system, and saved to the sample file directory	All assertions are met and passed by backend unit test	Pass
TC2	F211	Browse all uploaded STL files in the sample file directory	System displays a list of all uploaded STL files in the sample file directory	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC3	F211	Browse all uploaded STL files, without ever uploading a STL file	System displays an empty list of uploaded STL files	All assertions are met and passed by backend unit test	Pass
TC4	F211	Provide a file with unsupported format (e.g. .txt, .jpg, .pdf) to system through file settings panel	System rejects the file and displays an error indicating unsupported format	All assertions are met and passed by backend unit test	Pass
TC5	F212	Open a previously uploaded STL file	System loads the STL file from the sample file directory	All assertions are met and passed by backend unit test	Pass

6.1.2 Model Configuration and Slicing

Table 2 gives the results for model configuration and slicing tests (TC6–TC12).

Table 2: Import Manager: Model Configuration and Slicing Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC6	F213	Specify custom voxel size	System applies the dimensions and slices the model into voxels of the requested size	All assertions are met and passed by backend unit test	Pass
TC7	F214	Specify custom geometric model size	System scales the model to desired model size before slicing	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC8	F216	Import a model containing more than MAX_VOXELS voxels.	System partitions the model into display sections not exceeding MAX_VOXELS	All assertions are met and passed by backend unit test	Pass
TC9	F216	Import a model containing less than MAX_VOXELS voxels	System returns only one partition with all voxels visible	All assertions are met and passed by backend unit test	Pass
TC10	F216	Delete a partition with no voxels	System deletes the partition and returns the model to the previous state	All assertions are met and passed by backend unit test	Pass
TC11	F213, F214, F215	Voxelize uploaded STL model into voxels	System voxelizes the model into voxels of the requested size using Triangle Meshes	All assertions are met and passed by backend unit test	Pass
TC12	F213, F214, F215	Voxelize non-existing STL model into voxels	System returns an error indicating the STL model does not exist	All assertions are met and passed by backend unit test	Pass

6.2 Visualization Manager Tests

6.2.1 3D Model Rendering and Display

Table 3 gives the results for 3D model rendering and display tests (TC13–TC16).

Table 3: Visualization Manager: 3D Model Rendering and Display Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC13	F221	Import a STL model into the system	System renders a non-voxelized 3D STL model using Three.js	All assertions are met and passed by frontend unit test	Pass
TC14	F221	Open a voxelized project from file directory	System renders a voxelized 3D model using Three.js broken down into layers	All assertions are met and passed by frontend unit test	Pass
TC15	F222	Open a voxelized project from file directory where the number of voxles is greater than MAX_VOXELS	System displays a partitions panel with more than 1 partition	All assertions are met and passed by frontend unit test	Pass
TC16	F222	Open a voxelized project from file directory where the number of voxles is less than MAX_VOXELS	System displays a partitions panel with only 1 partition	All assertions are met and passed by frontend unit test	Pass

6.2.2 Layer Focus and Voxel Selection Tests

Table 4 gives the results for layer focus and voxel selection tests (TC17–TC24).

Table 4: Visualization Manager: Layer Focus and Voxel Selection Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC17	F222	Pick a specific partition to focus on from the partitions panel.	The selected partition is isolated and displayed as a 3D view without any other partitions.	All assertions are met and passed by frontend unit test	Pass
TC18	F223	Rotate the model using interface controls.	Model rotates smoothly and intuitively	All assertions are met and passed by frontend unit test	Pass
TC19	F223	Zoom the model using interface controls.	Model zooms smoothly and intuitively	All assertions are met and passed by frontend unit test	Pass
TC20	F224	Select a specific layer to focus on from the 3D model display.	Selected layer is isolated, layer editor is opened displaying the layer voxels and properties in a 2D view.	All assertions are met and passed by frontend unit test	Pass
TC21	F225	Select individual voxel within the focused layer from the layer editor.	The selected voxel is highlighted in the 2D layer view	All assertions are met and passed by frontend unit test	Pass
TC22	F225	Select multiple voxels within the focused layer from the layer editor using lasso selection.	The selected voxels are highlighted in the 2D layer view	All assertions are met and passed by frontend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC23	F225	Select all voxels in the partition on the 3D view (CTRL+A)	All voxels in the partition are highlighted in the 3D view	All assertions are met and passed by frontend unit test	Pass
TC24	F225	Select all voxels in the layer on the 2D layer view (CTRL+A)	All voxels in the layer are highlighted in the 2D layer view	All assertions are met and passed by frontend unit test	Pass

6.2.3 Material and Magnetization Tracking

Table 5 gives the results for material and magnetization tracking tests (TC25–TC26).

Table 5: Visualization Manager: Material and Magnetization Tracking Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC25	F226	Assign material IDs to a voxel or multiple voxels in the layer editor.	The color of the voxels are updated to the assigned material color.	All assertions are met and passed by frontend unit test	Pass
TC26	F227	Assign magnetization to a voxel or multiple voxels in the layer editor.	The voxels assigned will be highlighted with a X-arrow icon indicating magnetization.	All assertions are met and passed by frontend unit test	Pass

6.3 Editing Manager Tests

The Editing Manager tests are organized into three subsections: editing a voxel (TC27–TC49), utilizing the history manager (TC50–TC58), and saving project (TC59–TC62).

6.3.1 Editing a voxel

Table 6 gives the results for voxel editing tests (TC27–TC49): magnetization and material assignment, layer access, and adding, deleting, or resetting voxels.

Table 6: Editing Manager: Editing a voxel (system and backend)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC27	F231	Update magnetization of a single voxel (e.g. x=1.0, y=0.0, z=0.0).	System saves the magnetization vector of the voxel to the database	All assertions are met and passed by backend unit test	Pass
TC28	F231	Update magnetization of a multiple voxels (e.g. x=1.0, y=0.0, z=0.0).	System saves the magnetization vector of the voxels to the database	All assertions are met and passed by backend unit test	Pass
TC29	F231	Update magnetization of a non-existing voxel (e.g. x=1.0, y=0.0, z=0.0).	System returns an error indicating the voxel does not exist	All assertions are met and passed by backend unit test	Pass
TC30	F231	Update magnetization of a voxel with invalid magnetization vector (e.g. x=1.0, y=0.0, z=0.0).	System returns an error indicating the magnetization vector is invalid	All assertions are met and passed by backend unit test	Pass
TC31	F232	Create a favourite magnetization vector (e.g. x=1.0, y=0.0, z=0.0).	System saves the magnetization vector to the favourite bar	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC32	F233	Assign material ID to a single voxel (e.g. 2).	System saves the material ID to the voxel and in the database	All assertions are met and passed by backend unit test	Pass
TC33	F233	Assign material ID to a multiple voxels (e.g. 2).	System saves the material ID to the voxels and in the database	All assertions are met and passed by backend unit test	Pass
TC34	F233	Assign material ID to a non-existing voxel (e.g. 2).	System returns an error indicating the voxel does not exist	All assertions are met and passed by backend unit test	Pass
TC35	F231, F233	Assign material ID to a voxel and magnetization to a voxel (e.g. 2, x=1.0, y=0.0, z=0.0).	System saves the material ID and magnetization vector to the voxel and in the database	All assertions are met and passed by backend unit test	Pass
TC36	F234	Assign label "Steel" to material ID 1.	Material ID 1 is associated with the label "Steel" in the database.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC37	F235	Copy assigned properties from one layer to another layer.	Target layer voxels receive the same material and magnetization properties as the source.	All assertions are met and passed by backend and frontend unit test	Pass
TC38	F238	Get list of layers in the project.	The list of layers is returned.	All assertions are met and passed by backend unit test	Pass
TC39	F238	Get specific layer in the project based on the layer index and axis.	The specific layer is returned.	All assertions are met and passed by backend unit test	Pass
TC40	F238	Get specific layer with non-existing layer index.	System returns an error indicating the layer index does not exist.	All assertions are met and passed by backend unit test	Pass
TC41	F238	Get specific layer with non-existing axis.	System returns an error indicating the axis does not exist.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC42	F238	Get specific layer with non-existing project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC43	F238	Get specific layer with non-existing partition.	System returns an error indicating the partition does not exist.	All assertions are met and passed by backend unit test	Pass
TC44	F239	Add a new voxel at new indices.	A new voxel is added to the database at the new indices with default material and magnetization.	All assertions are met and passed by backend and frontend unit test	Pass
TC45	F239	Delete an existing voxel.	A voxel is removed from the database at the existing indices.	All assertions are met and passed by backend and frontend unit test	Pass
TC46	F239	Delete a non-existent voxel.	System returns an error indicating the voxel does not exist.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC47	F2310	Reset materials on selected voxels.	The voxels are reset to the default material in the database.	All assertions are met and passed by backend and frontend unit test	Pass
TC48	F2310	Reset magnetizations on selected voxels.	The voxels are reset to the default magnetization in the database.	All assertions are met and passed by backend and frontend unit test	Pass
TC49	F2310	Reset materials and magnetizations on selected voxels.	The voxels are reset to the default material and magnetization in the database.	All assertions are met and passed by backend and frontend unit test	Pass

6.3.2 Utilizing the history manager

Table 7 gives the results for history manager tests (TC50–TC58): undo, redo, and clear history.

Table 7: Editing Manager: Utilizing the history manager (backend)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC50	F237	Undo any updates made to a voxel.	The changes are undone and the voxel returns to the previous state.	All assertions are met and passed by backend unit test	Pass
TC51	F237	Redo any undone updates to a voxel.	The changes are redone and the voxel returns to the previous state.	All assertions are met and passed by backend unit test	Pass
TC52	F237	Undo any added voxels.	The voxels are removed from the database.	All assertions are met and passed by backend unit test	Pass
TC53	F237	Redo any deleted voxels.	The voxels are added back to the database.	All assertions are met and passed by backend unit test	Pass
TC54	F237	Undo any reset material on a voxel.	The voxel is reset to the default material.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC55	F237	Redo any reset material on a voxel.	The voxel is reset to the default material.	All assertions are met and passed by backend unit test	Pass
TC56	F237	Undo when history empty.	System returns an error indicating the history is empty	All assertions are met and passed by backend unit test	Pass
TC57	F237	Redo when redo stack empty.	System returns an error indicating the redo stack is empty	All assertions are met and passed by backend unit test	Pass
TC58	F237	Clear history.	Both history and redo stacks empty.	All assertions are met and passed by backend unit test	Pass

6.3.3 Saving project

Table 8 gives the results for saving project tests (TC59–TC62).

Table 8: Editing Manager: Saving project (backend)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC59	F236	Save the project automatically.	The project is saved to the database.	All assertions are met and passed by backend unit test	Pass
TC60	F2311	Save the project manually.	The project is saved to the database.	All assertions are met and passed by backend unit test	Pass
TC61	F2311	Save a non-existent project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC62	F2311	Save a project with invalid data.	System returns an error indicating the project data is invalid.	All assertions are met and passed by backend unit test	Pass

6.4 Export Manager Tests

6.4.1 Export Validation and File Export

Table 9 gives the results for export validation and file export (TC63–TC72).

Table 9: Export Manager Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC63	F241	Validate all voxels have magnetization.	System checks if all voxels have magnetization and returns an error if any voxels are missing magnetization.	All assertions are met and passed by backend unit test	Pass
TC64	F241	Validate all voxels have material.	System checks if all voxels have material and returns an error if any voxels are missing material.	All assertions are met and passed by backend unit test	Pass
TC65	F242	Export CSV file of an existing project.	System exports a CSV file with all voxel metadata.	All assertions are met and passed by backend unit test	Pass
TC66	F242	Export CSV file of a non-existent project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC67	F242	Export CSV file of a project with invalid data.	System returns an error indicating the project data is invalid.	All assertions are met and passed by backend unit test	Pass
TC68	F243	Export project file of an existing project.	System exports a project file with all project data.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC69	F243	Export project file of a non-existent project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC70	F243	Export project file of a project with invalid data.	System returns an error indicating the project data is invalid.	All assertions are met and passed by backend unit test	Pass
TC71	F245	Export model with summary option enabled.	System exports a model file with a summary of the model.	All assertions are met and passed by backend unit test	Pass
TC72	F241	Export operation is interrupted (simulated failure).	Original project data remains intact and unaltered.	All assertions are met and passed by backend unit test	Pass

7 Changes Due to Testing

During the VnV process, several changes to the project that were made based on the feedback from the stakeholders (supervisor and graduate students) that will be used to improve the system. These changes are related to the usability and user input, interaction flow, and performance adjustments.

7.1 Usability and User Input Adjustments

Feedback from usability testing led to the following usability and user-input changes:

- **Project and file menus:** A project popup after selecting a file to show existing projects, improving feedback and selection, as it was not clear where to naviagte to open a project.
- **Layer editor:** The magnitude control in the layer editor was removed to reduce clutter and confusion and as it was not used by the stakeholders.

- **File and material naming:** File naming was reviewed to handle spaces and to indicate when a file with the same name already exists. Material naming in settings was reviewed for clarity and consistency.
- **Magnetization display:** A checkmark or toggle was added to show or hide magnetization, giving users control over what is displayed.
- **Partition behaviour:** Default partition centring was changed so partitions are not automatically centred. A rename-partition capability was added.

7.2 Interaction Flow Adjustments

The following changes address the sequence of user actions and overall flow from opening a file to working with partitions:

- **Partition visibility:** The cause of partitions not showing immediately was investigated and addressed so that partitions appear promptly after selecting a project or confirguring a new one.
- **Reduced redundancy:** When a file is already open, re-selecting or re-confirming the same file is no longer required for subsequent actions where possible.

7.3 Performance Adjustments

Performance feedback from testing led to optimisations focused on model loading and rendering:

- **Flow from first open to partitions:** The slowness of the flow from first opening a file to obtaining partitions was addressed through performance improvements in file loading and initial processing.
- **Model rendering:** Trimesh (or equivalent mesh handling) was used or integrated to speed up model rendering, reducing latency when displaying the voxelised model and partitions.

7.4 Future Improvements

Usability testing also highlighted potential enhancements for future work:

- **Partition logic:** Revisit or extend partition logic to support more flexible partitioning strategies and user control.
- **Lasso tool:** Consider a lasso (or similar) selection tool to add voxels in a more intuitive way.
- **Export Model Summary:** Add a model summary to the export model file to allow users to see what changes were made to the model.

- **Export Progress Trackers:** Test the export performance of the system to ensure it meets the requirements and improve the progress tracker.

Overall, the testing phase was successfull and played a crucial role in the development of the system. It allowed for the identification of issues and the implementation of improvements, ensuring the system is of high quality and meets the requirements of the stakeholders.

8 Automated Testing

All tests for the Python backend and for the TypeScript frontend modules are automated. The pass/fail results in the tables in Section 6 are backed by the output of these test runs, readers can reproduce the evidence by running the tests as described in the VnV Plan.

- **Backend (Python/PyTest):** From the project root, run `pytest` in `src/backend/` (e.g. `cd src/backend && pytest`). The console output shows per-test pass/fail and any failures; this output is the evidence for the "Actual Result" and "Result" columns in the tables in Section 6.
- **Frontend (TypeScript/Jest):** From `src/frontend/`, run `npm run test`. The console (or test report) output is the evidence for the "Actual Result" and "Result" columns in the tables in Section 6.

System-level and UI tests are executed manually; their outcomes are recorded in the corresponding tables in Section 6 and match the expected results listed there.

9 Trace to Requirements

The test cases in this report are the execution of the system and unit tests defined in the VnV Plan. The following tables show the traceability between these test cases and the functional requirements from the SRS. A filled cell (■) indicates that the test case verifies the corresponding requirement; an empty cell (□) indicates it does not.

9.1 Functional Requirements

Table 10 shows the trace between Import Manager test cases and corresponding requirements.

Table 10: Trace Between Import Manager Test Cases and Corresponding Requirements

Reqs:	F211	F212	F213	F214	F215	F216
TC1	■	□	□	□	□	□
TC2	■	□	□	□	□	□
TC3	■	□	□	□	□	□
TC4	■	□	□	□	□	□
TC5	□	■	□	□	□	□
TC6	□	□	■	□	□	□
TC7	□	□	□	■	□	□
TC8	□	□	□	□	□	■
TC9	□	□	□	□	□	■
TC10	□	□	□	□	□	■
TC11	□	□	■	■	■	□
TC12	□	□	■	■	■	□

Table 11 shows the trace between Visualization Manager test cases and corresponding requirements.

Table 11: Trace Between Visualization Manager test cases and corresponding requirements

Reqs:	F221	F222	F223	F224	F225	F226	F227
TC13	■	□	□	□	□	□	□
TC14	■	□	□	□	□	□	□
TC15	□	■	□	□	□	□	□
TC16	□	■	□	□	□	□	□
TC17	□	■	□	□	□	□	□
TC18	□	□	■	□	□	□	□
TC19	□	□	■	□	□	□	□
TC20	□	□	□	■	□	□	□
TC21	□	□	□	□	■	□	□
TC22	□	□	□	□	■	□	□
TC23	□	□	□	□	■	□	□
TC24	□	□	□	□	■	□	□
TC25	□	□	□	□	□	■	□
TC26	□	□	□	□	□	□	■

Table 12 shows the trace between Editing Manager test cases and corresponding require-

ments (Part 1: voxel editing and history).

Table 12: Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 1: voxel editing and history)

Reqs:	F231	F232	F233	F234	F235	F237
TC27	■	□	□	□	□	□
TC28	■	□	□	□	□	□
TC29	■	□	□	□	□	□
TC30	■	□	□	□	□	□
TC31	□	■	□	□	□	□
TC32	□	□	■	□	□	□
TC33	□	□	■	□	□	□
TC34	□	□	■	□	□	□
TC35	■	□	■	□	□	□
TC36	□	□	□	■	□	□
TC37	□	□	□	□	■	□
TC38	□	□	□	□	□	■
TC39	□	□	□	□	□	■
TC40	□	□	□	□	□	■
TC41	□	□	□	□	□	■
TC42	□	□	□	□	□	■
TC43	□	□	□	□	□	■
TC44	□	□	□	□	□	■
TC45	□	□	□	□	□	■
TC46	□	□	□	□	□	■

Table 13 shows the trace between Editing Manager test cases and corresponding requirements (Part 2: layers, add/delete, reset, save).

Table 13: Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 2: layers, add/delete, reset, save)

Reqs:	F238	F239	F2310	F2311
TC47	■	□	□	□
TC48	■	□	□	□
TC49	■	□	□	□
TC50	■	□	□	□
TC51	■	□	□	□
TC52	■	□	□	□
TC53	□	■	□	□
TC54	□	■	□	□
TC55	□	■	□	□
TC56	□	□	■	□
TC57	□	□	■	□
TC58	□	□	■	□
TC59	□	□	□	■
TC60	□	□	□	■
TC61	□	□	□	■
TC62	□	□	□	■

Table 14 shows the trace between Export Manager test cases and corresponding requirements.

Table 14: Trace Between Export Manager Test Cases and Corresponding Requirements

Reqs:	F241	F242	F243	F244	F245
TC63	■	□	□	□	□
TC64	■	□	□	□	□
TC65	□	■	□	□	□
TC66	□	■	□	□	□
TC67	□	■	□	□	□
TC68	□	□	■	□	□
TC69	□	□	■	□	□
TC70	□	□	■	□	□
TC71	□	□	□	□	■
TC72	■	□	□	□	□

9.2 Non-Functional Requirements

Table 15 shows the trace between Non-Functional Requirements Import Manager test cases and corresponding requirements.

Table 15: Trace Between Non-Functional Requirements Import Manager Test Cases and Corresponding Requirements

Reqs:	NF211	NF212	NF213
TC8	■	□	□
TC9	■	□	□
Not Explicitly Tested	□	■	□
Tested by usability test (Graduate Students)	□	□	■

Table 16 shows the trace between Non-Functional Requirements Visualization Manager test cases and corresponding requirements.

Table 16: Trace Between Non-Functional Requirements Visualization Manager Test Cases and Corresponding Requirements

Reqs:	NF221	NF222	NF223
TC18	■	□	□
TC19	■	□	□
TC15	□	■	□
TC16	□	■	□
Not Explicitly Tested	□	□	■

Table 17 shows the trace between Non-Functional Requirements Editing Manager test cases and corresponding requirements.

Table 17: Trace Between Non-Functional Requirements Editing Manager Test Cases and Corresponding Requirements

Reqs:	NF231	NF232
TC59	■	□
Tested by usability test (Graduate Students)	□	■

Table 18 shows the trace between Non-Functional Requirements Export Manager test cases and corresponding requirements.

Table 18: Trace Between Non-Functional Requirements Export Manager Test Cases and Corresponding Requirements

Reqs:	NF241	NF242	NF243
TC63	■	■	□
TC64	■	■	□
TC65	■	■	□
TC66	■	■	□
TC67	■	■	□
Not Explicitly Tested	□	□	■

10 Trace to Modules

The following tables show the traceability between the test cases in this report and the modules defined in the Module Guide. A filled cell (■) indicates that the test case exercises or verifies behaviour involving that module; an empty cell (□) indicates it does not. The module identifiers correspond to the modules in the MG.

Table 19 shows the trace between Import Manager test cases and modules.

Table 19: Trace Between Import Manager Test Cases and Modules

TC	M1	M2	M3	M4	M5
TC1	■	□	□	■	□
TC2	■	□	□	■	□
TC3	■	□	□	■	□
TC4	■	□	□	■	□
TC5	■	□	□	■	□
TC6	■	■	□	□	■
TC7	■	□	□	□	■
TC8	□	□	■	□	□
TC9	□	□	■	□	□
TC10	□	□	■	□	□
TC11	■	■	□	■	□
TC12	■	■	□	■	□

Table 20 shows the trace between Visualization Manager test cases and modules.

Table 20: Trace Between Visualization Manager Test Cases and Modules

TC	M3	M5	M6	M9	M10	M14	M15
TC13	☐	☐	■	☐	☐	☐	☐
TC14	☐	☐	■	☐	☐	☐	☐
TC15	■	■	■	☐	☐	■	☐
TC16	■	■	■	☐	☐	■	☐
TC17	■	■	■	☐	☐	■	☐
TC18	☐	■	■	☐	☐	■	■
TC19	☐	■	■	☐	☐	■	■
TC20	☐	■	■	☐	☐	■	■
TC21	☐	■	■	■	■	■	■
TC22	☐	■	■	■	■	■	■
TC23	☐	■	■	■	■	■	■
TC24	☐	■	■	■	■	■	■
TC25	☐	☐	■	■	■	■	■
TC26	☐	☐	■	■	■	■	■

Table 21 shows the trace between Editing Manager test cases and modules (Part 1: voxel editing and history).

Table 21: Trace Between Editing Manager Test Cases and Modules (Part 1: voxel editing and history)

TC	M5	M7	M8	M9	M10	M13	M14
TC27	■	■	□	□	□	□	■
TC28	■	■	□	□	□	□	■
TC29	■	■	□	□	□	□	■
TC30	■	■	□	□	□	□	■
TC31	■	■	□	□	■	□	□
TC32	■	■	□	□	□	□	■
TC33	■	■	□	□	□	□	■
TC34	■	■	□	□	□	□	■
TC35	■	■	□	□	□	□	■
TC36	■	■	□	□	■	□	□
TC37	■	■	■	■	□	□	■
TC38	■	■	■	□	□	□	□
TC39	■	■	■	□	□	□	□
TC40	■	■	■	□	□	□	□
TC41	■	■	■	□	□	□	□
TC42	■	■	■	□	□	□	□
TC43	■	■	■	□	□	□	□
TC44	■	■	■	□	□	□	□
TC45	■	■	■	□	□	□	□
TC46	■	■	■	□	□	□	□

Table 22 shows the trace between Editing Manager test cases and modules (Part 2: layers, add/delete, reset, save).

Table 22: Trace Between Editing Manager Test Cases and Modules (Part 2: layers, add/delete, reset, save)

TC	M4	M5	M7	M10	M14
TC47	☐	■	■	■	■
TC48	☐	■	■	■	■
TC49	☐	■	■	■	■
TC50	☐	■	■	■	■
TC51	☐	■	■	■	■
TC52	☐	■	■	■	■
TC53	☐	■	■	☐	■
TC54	☐	■	■	☐	■
TC55	☐	■	■	☐	■
TC56	☐	■	■	☐	■
TC57	☐	■	■	☐	■
TC58	☐	■	■	☐	■
TC59	■	☐	☐	☐	☐
TC60	■	☐	☐	☐	☐
TC61	■	☐	☐	☐	☐
TC62	■	☐	☐	☐	☐

Table 23 shows the trace between Export Manager test cases and modules.

Table 23: Trace Between Export Manager Test Cases and Modules

TC	M9	M11	M12	M13	M14	M16
TC63	■	■	☐	☐	■	☐
TC64	■	■	☐	☐	■	☐
TC65	☐	☐	■	☐	■	■
TC66	☐	☐	■	☐	■	■
TC67	☐	☐	■	☐	■	■
TC68	☐	☐	■	☐	■	☐
TC69	☐	☐	■	☐	■	☐
TC70	☐	☐	■	☐	■	☐
TC71	☐	☐	■	☐	■	■
TC72	■	■	☐	☐	■	☐

11 Code Coverage Metrics

11.1 Python Code Coverage

The following section analyzes the code coverage of the Python code in the backend. The analysis is provided in the Figure 1 and is a measure of how much of the code is being tested, and helps us understand which parts of the code are not being tested.

Coverage report: 90%

Files Functions Classes

filter...

hide covered

coverage.py v7.13.4, created at 2026-03-08 20:10 -0400

File ▲	statements	missing	excluded	coverage
app/__init__.py	0	0	0	100%
app/config.py	15	1	0	93%
app/models/__init__.py	2	0	0	100%
app/models/schemas.py	44	0	0	100%
app/routers/__init__.py	0	0	0	100%
app/routers/edit_router.py	144	11	0	92%
app/routers/export_router.py	29	0	0	100%
app/routers/project_router.py	70	27	0	61%
app/routers/stl_router.py	41	7	0	83%
app/services/__init__.py	0	0	0	100%
app/services/export_service.py	21	0	0	100%
app/services/history_management_service.py	39	4	0	90%
app/services/mesh_service.py	6	3	0	50%
app/services/model_editing_service.py	38	0	0	100%
app/services/model_structure_service.py	98	6	0	94%
app/services/model_tracking_service.py	51	0	0	100%
app/services/partition_manager.py	39	0	0	100%
app/services/project_management_service.py	36	5	0	86%
app/services/voxel_service.py	7	3	0	57%
Total	**680**	**67**	**0**	**90%**

coverage.py v7.13.4, created at 2026-03-08 20:10 -0400

Figure 1: Python Code Coverage

The backend codebase has an overall code coverage of 90% for statements, branches, functions, and lines. The code coverage overall met the VnV Plan's requirement of 50%. The `voxel_service.py` was the file that was the most difficult to test because it utilizes the `trimesh` library for voxelization. The team will furthur analyze the code coverage of the `voxel_service.py` file to understand which parts of the code are not being tested and improve the coverage of the file.

11.2 TypeScript Code Coverage

The following section analyzes the code coverage of the TypeScript code in the frontend. The analysis is provided in the Figure 2 and is a measure of how much of the code is being tested, and helps us understand which parts of the code are not being tested.

```
-----------------------|---------|----------|---------|---------|-------------------------------
File                   | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------------------|---------|----------|---------|---------|-------------------------------
All files              |   83.08 |    65.98 |   85.39 |   84.14 |
 src                   |   84.41 |       50 |   79.36 |   85.32 |
  App.tsx              |   84.41 |       50 |   79.36 |   85.32 | 113,132,157-163,169-170,195-197,
 src/components        |    72.9 |    66.74 |   88.18 |   73.47 |
  Footer.tsx           |     100 |      100 |     100 |     100 |
  LayerEditor.tsx      |   58.56 |     46.8 |   76.59 |   59.19 | 111-114,133-136,146-148,155-167,
  MenuBar.tsx          |     100 |    96.77 |     100 |     100 | 116,290,338
  ModelSelector.tsx    |     100 |      100 |     100 |     100 |
  NewProjectDialog.tsx |   86.31 |    78.26 |   90.47 |   88.37 | 91-102,238
  PartitionsPanel.tsx  |     100 |    94.11 |     100 |     100 | 33
  ProjectSelector.tsx  |     100 |      100 |     100 |     100 |
  StatusMessage.tsx    |     100 |      100 |     100 |     100 |
  UploadButton.tsx     |      75 |     62.5 |     100 |   72.72 | 12,16-17
  UploadMessage.tsx    |     100 |      100 |     100 |     100 |
 src/utils             |   97.04 |    78.81 |   86.79 |   99.69 |
  api.ts               |   96.37 |    80.32 |   81.08 |     100 | 38,53-58,106,157,173,218,326-327
  threeUtils.ts        |   97.76 |    77.19 |     100 |   99.37 | 293
-----------------------|---------|----------|---------|---------|-------------------------------

============================== Coverage summary ==============================
Statements   : 83.08% ( 1022/1230 )
Branches     : 65.98% ( 452/685 )
Functions    : 85.39% ( 193/226 )
Lines        : 84.14% ( 950/1129 )
==============================================================================

Test Suites: 16 passed, 16 total
Tests:       221 passed, 221 total
Snapshots:   0 total
Time:        4.317 s, estimated 6 s
Ran all test suites.
```

Figure 2: TypeScript Code Coverage

The frontend codebase has an overall code coverage of 83.08% for statements, 65.98% for branches, 85.39% for functions, and 84.14% for lines. The code coverage overall met the VnV Plan's requirement of 50%. The team will continue to improve the coverage of the frontend codebase in the future.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

Omar:

Daniel: I worked on the backend unit tests, as well as the organization of the usability test with our supervisor and her graduate students, which both went fairly smoothly in spite of the time crunch. The reorganization of the backend into cohesive modules and functions that I participated in was extremely helpful when generating all unit test cases, as well as their relative simplicity. The structure of the usability test - providing as little instruction as possible - also made it easy to identify parts of the software that still need work in terms of understandability.

Andrew: I worked on UI unit tests, listing all functional requirements in the report, and explaining the alternative COMSOL to AutoVox. Writing the UI tests went well, and I learned a lot about mocking modules, and making sure functions that needed to be fired and validated helped find any errors in the codebase.

Olivia: During this deliverable, I helped write some of the backend unit tests and was the primary person writing the usability report after usability testing, which was conducted by all group members. Due to the nature of this deliverable, I think that it really helped us narrow down some of the major areas that require improvement and further development as well as build confidence in the code that was already developed. With the final deadlines quickly approaching, there is a lot to do in very little time. Therefore, this type of direction is important to ensure that we make the best use out of the time we have remaining.

Khalid: I worked on the unit tests and tracibility between the test cases and the requirements. Having the tracibility between the functional requirements and modules in the MIS document was very helpful in ensuring that all the test cases were covered and that the

test cases were testing the correct functionality. It was also helpful to have the tracibility between the test cases and the modules in the VnV Plan document. Overall, this deliverable went really well as everything was clearly defined and we were able to complete it on time. Another thing that was helpful was the fact that I didn't have much university work to do at the same time, so I was able to focus on this deliverable and get it done on time.

2. What pain points did you experience during this deliverable, and how did you resolve them?

Omar:

Daniel: Less of a pain point and more a frustration due to lack of experience; when generating unit tests my primary struggle was learning proper use of pytest and mocking classes, as well as when to mock. While I had planned to become familiar with it prior to needing to write these tests, other commitments prevented me from doing so, and I had to learn on the fly how to accomplish what I wanted to do. Fortunately, it was less complex once I figured out the syntax and I was able to finish all unit tests by the deadline I promised.

Andrew: I didn't experience many pain points, however it was hard to see exactly what our report should look like. The rubric wasn't that detailed, and the only way to see what to do correctly was to check prior submissions on the GitLab.

Olivia: In relation to the usability testing, one of the challenges that we experienced was choosing exactly how much guidance to give the user as they interacted with the software. We want to be able to provide enough structure and context so that the user is able to understand what we want them to do and how they can do it, while not making instructions too explicit in a way that compromises how a user would intuitively interact with and navigate through the software. While we went into the usability testing with a plan, things were definitely adapted in the moment. I think this is a natural part of usability testing. While you can have an idea on how you think people will interact with something, reality isn't always what you imagine. If we decide to conduct another round of usability testing within the coming weeks after further developing the software, there will definitely be changes in the structure and method we choose to conduct usability testing that reflect some of our observations during the first round.

Khalid: I didn't experience much pain points during this deliverable. The only pain point I had was just the amount of repetitive work that was required to document the test cases and the requirements. However, I was able to resolve it by breaking down the work into smaller tasks and completing them one by one.

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

The team: Several parts of this document came from discussions with our supervisor and with a proxy (our peers). The scope of **functional and non-functional testing**—what needed to be tested and how, was conducted by speaking to the proxy and jointly defining the test objectives and coverage. The **Comparison to Existing Implementation** section came from speaking to our supervisor and to the graduate students who use the lab's 3D printing workflow. Those conversations helped us understand how they currently complete their objectives (e.g., with COMSOL) and what AutoVox should improve or replace. **Unit testing**, **automated testing**, and **coverage** were driven mainly by the proxy, we relied on peer discussion and shared standards to decide frameworks (e.g., PyTest, Jest), what to automate, and what coverage targets to aim for. **Changes due to testing** came from a mix of both: the proxy helped prioritize and validate technical changes, while the supervisor and graduate students provided context on whether those changes aligned with real usage and research needs.

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

We did not make any major changes to the VnV Plan, the activities we conducted aligned with what was planned. We were able to predict a feasible amount of effort and the right tasks mainly because the plan was well scoped and supported by strong traceability. The traceability between the functional requirements (SRS), the modules (MIS and Module Guide), and the test cases in the VnV Plan made it clear which tests were needed and what they had to cover, so we could estimate effort and avoid gaps or duplicate work. The backend had also been reorganized into cohesive modules and functions before we wrote the unit tests, which made the test cases straightforward to implement and kept the workload manageable. For usability testing, the plan specified the overall approach (e.g., exploratory testing with minimal instruction and a task-based evaluation) but left out the exact list of tasks. When we ran the usability test, we did refine the specific tasks and the level of guidance given to participants (e.g. how much context to provide before each task). These were minor adjustments rather than changes to the plan itself, and they did not affect the planned objectives or deliverables. In future projects, we would again rely on clear traceability and a plan that fixes the strategy and scope while allowing small operational details (like concrete usability tasks) to be finalized at execution time.